DAEMONGUARD

A GREENOPS-BASED INTELLIGENT RESOURCE OPTIMIZATION SYSTEM FOR WINDOWS

CO2307 - SOFTWARE ENGINEERING
MINI PROJECT PROPOSAL

Group No. : 12

Group Members : R.M.D.N. Bandara - 24/ENG/020

R.N. Keshan - 24/ENG/119

W.K.P.M. Thilakarathne - 24/ENG/152

S.D.A. Mudalige - 24/ENG/129

Date : 12.10.2026

1. INTRODUCTION

Windows systems today run foreground applications, background processes, and system services in parallel. All of them require CPU, memory, disk space, network access, and graphics processing power. Some could continue using large amounts of resources without performing any work at all, leading to inefficiency and unnecessary power consumption. Windows Task Manager gives you a chance to monitor them, but it's hard to tell which processes are not useful at all. While enterprise cloud infrastructures widely employ GreenOps and FinOps frameworks to govern carbon footprints and energy expenditure, local client-side workstations lack lightweight and transparent tools dedicated to managing runtime compute waste.

DaemonGuard is an intelligent, local GreenOps resource optimization system designed specifically for Windows environments integrating multi-metric telemetry analysis with built-in system safety and explainable user recommendations.

2. PROBLEM DEFINITION

Applications, background services and user daemons continuously draw power across multiple hardware vectors, specifically monitoring CPU and RAM for performance pressure and background activity, disk and inactive memory usage, network connection activity and GPU idle times. Although built-in utilities such as Windows Task Manager expose raw telemetry, they act purely as passive data aggregators which requires users to manually diagnose whether a process is necessary or actively wasting resources.

Conventional automated cleanup tools rely on rigid and single-variable thresholds, for instance, flagging any process exceeding a static memory ceiling. This primitive logic generates high false-positive rates that risk the accidental termination of legitimate background workloads like active compiler threads or rendering tasks.

Core Research Question: Can potentially wasteful background processes be identified safely and transparently while supporting GreenOps-based local optimization without disrupting core operating system stability?

3. MAIN OBJECTIVE

DaemonGuard aims to bridge the gap between enterprise sustainability frameworks and local operating system management by delivering an explainable, human-in-the-loop advisory dashboard.

The primary objectives include:

1. Multi-Metric Correlative Analysis: Moving beyond primitive single-variable checks by correlating co-dependent signals (CPU activity, memory footprint, disk/network I/O and user inactivity duration).

2. Explainable Governance (0-10 Waste Score): Translating raw performance metrics into a transparent scoring matrix so users understand the exact diagnostic reasoning behind every recommendation.

3. Operational Risk Decoupling: Enforcing robust safety guardrails that separate resource optimization scores from an absolute system risk index, protecting critical Windows services from accidental termination

4. CHARACTERISTICS AND SYSTEM ARCHITECTURE

4.1 The Five-Stage Processing Pipeline:

DaemonGuard operates as an active analysis engine structured around a transparent, user-controlled pipeline:

1. Resource Collector: Utilizes native C#/.NET diagnostic libraries and performance counters to continuously sample CPU utilization, memory allocation, disk activity, network connections, GPU utilization, and user idle times.

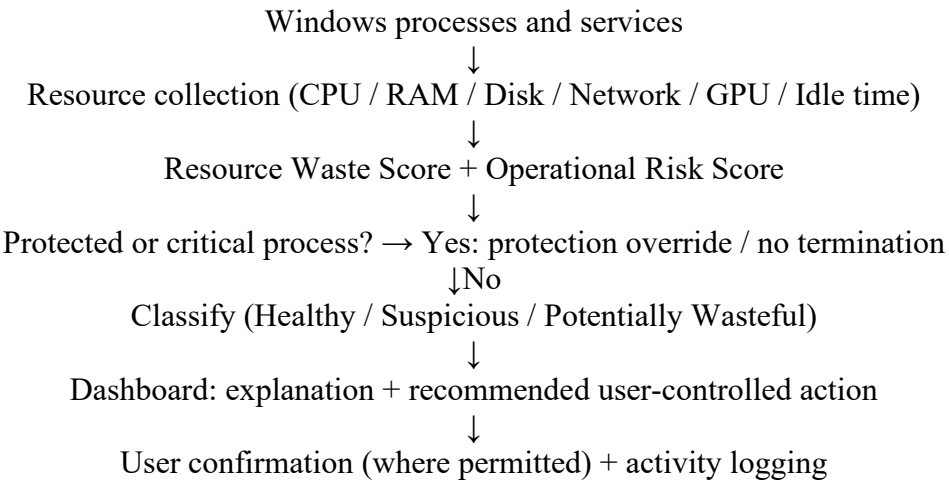
2. Analysis Engine: Processes raw telemetry through a heuristic multi-criteria decision matrix to calculate both a Resource Waste Score (0-10) and an Operational Risk Score.

3. Classification Layer: Categorizes monitored workloads into three distinct operational states: Healthy (0-3), Suspicious (4-6) or Potentially Wasteful (7-10).

4. Recommendation Output: Generates clear, actionable next steps for the user, including options to Ignore, Optimize, Restart or Close. Control remains strictly with the user as the system operates on a recommendation-based model rather than unmonitored automatic termination.

5. Dashboard UI: Displays real-time metrics, historical trends, and transparent score breakdowns.

4.2 Proposed processing flow



4.3 Explainable Intelligence and Heuristic Scoring:

To ensure complete transparency, DaemonGuard avoids black-box automation by mapping specific performance signals to an explicit point-scoring matrix:

- High Memory Usage: +3 points
- Long Inactivity Duration: +3 points
- Very Low CPU Usage: +2 points

- No Network Activity: +1 point
- No Disk Activity: +1 point

Example: A background process consuming 4 GB of RAM with negligible CPU activity, zero disk/network I/O and 40 minutes of continuous user inactivity accrues a high waste score. Crucially, achieving a high waste score never triggers automatic termination as it merely surfaces an advisory recommendation to the user.

5. KEY CHALLENGES AND SAFETY OVERRIDE ARCHITECTURE

5.1 System Challenges:

- **Differentiating Heavy Active Workloads:** Long-running production tasks such as 3D rendering in Blender or video encoding consume high CPU or RAM while appearing inactive to basic scripts. DaemonGuard inspects active disk write I/O and sub-process thread activity before flagging a process as wasteful.
- **Coexisting with OS Predictive Caching:** Windows utilizes internal optimization mechanisms (SysMain/Superfetch) to stage applications in memory for rapid launch. DaemonGuard functions exclusively as an advisory, human-in-the-loop dashboard rather than an aggressive process killer, ensuring built-in OS memory management is fully respected.

5.2 Safety Override Protocol:

To guarantee that optimization never compromises stability, critical operational risk overrides any high waste score. The internal protection logic executes a strict six-step safety sequence:

- Identifies Windows system processes and background services.
- Verifies the process owner and executable file path.
- Maintains an immutable protected-process exclusion list.
- Calculates a comprehensive Operational Risk Score.
- Disables termination capabilities for all critical system processes.
- Requires explicit user or administrator confirmation before executing any user-selected mitigation.

Through this, DaemonGuard highlights avoidable CPU time, inactive memory, inefficient background usage and potential CPU-minutes or memory-hours saved, laying the groundwork for future energy and carbon estimation modules.

6. METHODOLOGY AND PROPOSED EVALUATION

To validate the effectiveness and safety of the prototype, DaemonGuard will be evaluated across three core pillars as detection accuracy, false-positive rate and monitoring overhead.

6.1 Research Design and Baseline Establishment:
To validate the effectiveness and safety of DaemonGuard, the project will employ an proper evaluation framework. The primary baseline for comparison will be traditional static CPU and RAM threshold detection models such as those found in default administrative scripts.

6.2 Experimental Test Scenarios:
Testing will be conducted across structured operational environments to measure system resilience and accuracy which includes:

- Active high-memory user applications
- Idle high-memory background processes
- Standard background services and system daemons
- Core Windows protected system processes
- Simulated abnormal or runaway zombie workloads

6.3 Evaluation Metrics:
Performance and safety will be assessed simultaneously using four primary metrics:

- Detection Accuracy: The ratio of correctly identified wasteful processes versus missed zombie tasks.
- False-Positive Rate: The frequency with which active, legitimate tasks such as background renders or updates are incorrectly flagged.
- Detection Time: The latency between process resource stagnation and heuristic score generation.
- Monitoring Overhead: The CPU and memory footprint consumed by DaemonGuard's own background diagnostic engine.

7. RESOURCES

Resource Category	Technologies and Tools
Operating System (OS)	Windows 11
Programming Language and Framework	C# and .NET
User Interface	Windows Desktop UI
Data Storage	SQLite
Development Environment	Visual Studio and Git

8. PROJECT PLAN

Project Phase	W1-W2	W3-W4	W5-W6	W7-W8
Requirements Analysis and Architecture Design				
C# Telemetry Collection and Performance Wrapper				
Heuristic Waste Scoring and Safety Gaurdrails				
Dashboard UI and Advisory Control Integration				
System Evaluation, Stress testing and Fine-Tuning				
Final Documentation and Project Submission				

9. REFERENCES

[1] M. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations, 2016, doi: 10.18653/v1/n16-3020.

[2] P. Lago, N. Condori Fernandez, I. Fatima, M. Funke, and I. Malavolta, "The Sustainability Assessment Framework Toolkit: A Decade of Modeling Experience," Software and Systems Modeling, vol. 24, no. 2, pp. 361–383, Nov. 2024, doi: 10.1007/s10270-024-01230-9.

[3] R. Verdecchia, P. Lago, C. Ebert, C. de Vries, and C. Ebert, "Green IT and Green Software," IEEE Software, vol. 38, no. 6, pp. 7–15, Nov. 2021, doi: 10.1109/ms.2021.3102254.

[4] "System.Diagnostics Namespace," MicrosoftLearn. Accessed: Oct. 12, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics?view=net-11.0-pp